Data Structures (CSN12101)
ASSIGNMENT-2

Name : Aayansh Niranjan Date : 13/01/2026
Reg_No.: 20253004

Q1. Suppose you are a software developer working on an application for a library. The application maintains a list of books, each with a unique identification number. The library wants to implement a feature that displays books in increasing order of their ID numbers. To achieve this, the library has requested you to implement an Insertion Sort algorithm to sort the array of book IDs. Also analyze the run time complexity of the algorithms.

A1. CODE:

```
#include<stdio.h>
int main(){
  int n,i,j,key,a[100];
  scanf("%d",&n);
  for(i=0;i<n;i++) scanf("%d",&a[i]);
  for(i=1;i<n;i++){
    key=a[i];j=i-1;
    while(j>=0 && a[j]>key){a[j+1]=a[j];j--;}
    a[j+1]=key;
  }
  for(i=0;i<n;i++) printf("%d ",a[i]);
  printf("\n");
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_2$ ./p1
5
66 42 12 69 55
12 42 55 66 69
```

Q2. You are working as a software developer in a company that processes transaction amounts for various departments. Your task is to sort an array of transaction amounts in ascending order to help the finance team generate monthly reports more efficiently. The company prefers using Selection Sort because of its simplicity, but they need you to implement it and explain its run-time complexity.

A2. CODE:

```
#include<stdio.h>
int main(){
  int n,i,j,min,t,a[100];
  scanf("%d",&n);
  for(i=0;i<n;i++)scanf("%d",&a[i]);
  for(i=0;i<n-1;i++){
    min=i;
    for(j=i+1;j<n;j++) if(a[j]<a[min])min=j;
    t=a[i];
    a[i]=a[min];
    a[min]=t;
  }
  for(i=0;i<n;i++)printf("%d ",a[i]);
  printf("\n");
}
```

OUTPUT:

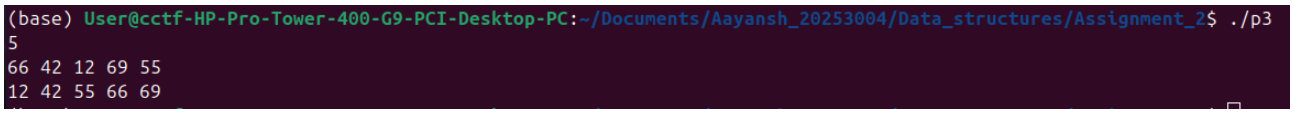
```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_2$ ./p2
5
66 42 12 69 55
12 42 55 66 69
```

Q3. You need to implement a recursive Selection Sort to sort customer transaction IDs in ascending order. The algorithm recursively finds the minimum element in the unsorted part, swaps it with the first unsorted element, and calls itself for the next unsorted portion. The run-time complexity of recursive Selection Sort is $O(n^2)$ in all cases because it performs n-1, n-2, ..., 1 comparisons, resulting in a quadratic time complexity. Also discuss the problem in case of large dataset is sorted using this approach.

A3. CODE:

```
#include<stdio.h>
int main(){
  int n,i,j,min,t,a[100];
  scanf("%d",&n);
  for(i=0;i<n;i++) scanf("%d",&a[i]);
  for(i=0;i<n-1;i++){
    min=i;
    for(j=i+1;j<n;j++)if(a[j]<a[min])min=j;
    t=a[i];
    a[i]=a[min];
    a[min]=t;
  }
  for(i=0;i<n;i++)printf("%d ",a[i]);
  printf("\n");
}
```

OUTPUT:



Q4.Write a C program to implement Heap sort in a given array of larger size. You also have to find the number of comparisons and time and space required to sort the array.

A4. CODE :

```
#include<stdio.h>
int main(){
   int n,i,j,t,a[100],c=0;
   scanf("%d",&n);
   for(i=0;i<n;i++) scanf("%d",&a[i]);
   for(i=1;i<n;i++){
      j=i;
      while(j>0 && a[(j-1)/2]<a[j]){
         t=a[j];
         a[j]=a[(j-1)/2];
         a[(j-1)/2]=t;
         j=(j-1)/2;
         c++;
      }
   }
   for(i=n-1;i>0;i--){
      t=a[0];
      a[0]=a[i];
      a[i]=t;
      j=0;
      while(2*j+1 < i){
         int k=2*j+1;
         if(k+1<i && a[k] < a[k+1])
            k++;
         if(a[j] < a[k]){
            t=a[j];
            a[j]=a[k];
            a[k]=t;
            j=k;
            c++;
         }
         else
            break;
      }
   }
   for(i=0;i<n;i++) printf("%d ",a[i]);
   printf("\nComparisons:%d",c);
   printf("\n");
}
```

OUTPUT :

